

Laborator 2

2.1 Arhitectura Intel x86

2.1.1 Structura unui sistem de calcul (considerații generale)

Majoritatea sistemelor de calcul din zilele noastre (și nu numai) sunt constituite din 3 tipuri de componente:

- memoria principală
- unitatea centrală de prelucrare (în engleză Central Processing Unit – CPU)
- dispozitive de intrare/ieșire

Memoria principală a sistemului este direct accesibilă de către unitatea centrală de prelucrare, și poate conține date sau cod (vom vedea ulterior că acesta este doar un tip mai special de date). Acest tip de memorie se mai numește și memorie RAM (Random Access Memory), deoarece orice locație aleatoare din aceasta, poate fi adresată în timp constant (spre deosebire de discurile magnetice, care favorizează citirea datelor secvențial, sau de memoriile asociative, unde datele sunt accesate prin conținut, nu prin adresă). O altă caracteristică a memoriei principale este volatilitatea. Atunci când alimentarea cu energie a sistemului este întreruptă, conținutul acesteia se pierde.

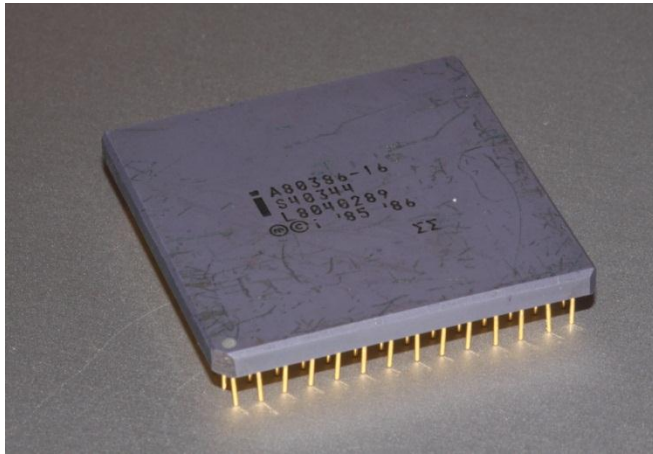
Unitatea centrală de prelucrare este un circuit care execută instrucțiunile dintr-un program, citind sau scriind date din memorie, efectuând diverse operații asupra acestora, sau accesând dispozitivele de intrare/ieșire.

Dispozitivele de intrare/ieșire fac diferența dintre un sistem de calcul și alte aparate care consumă energie electrică și produc căldură. Cele mai întâlnite astfel de dispozitive sunt tastatura, mouse-ul, monitorul, respectiv mediile de stocare. Prin acestea se asigură interacțiunea cu utilizatorul.

2.1.2 Familia Intel x86

Familia de procesoare x86 este, în prezent, cea mai folosită pe piața calculatoarelor desktop (în timp ce procesoarele ARM domină piața dispozitivelor mobile). Deși aceste procesoare au evoluat de-a lungul timpului, suferind diverse modificări arhitecturale, s-a pus un accent deosebit pe păstrarea compatibilității. Astfel, un program scris pentru generația '386 va rula fără probleme pe un procesor Intel de ultimă generație.

Procesoarele mai vechi au fost procesoare pe 16 bit, ceea ce însemna că registrele acestora (vom vedea ce înseamnă registru în secțiunea următoare) aveau o lungime de 16 bit, iar instrucțiunile erau mai scurte.



Generația a 3-a (80386) a adus procesoare pe 32 bit, în care regiștrii au lungimi pe 32 bit, iar setul de instrucțiuni este îmbogățit. Se păstrează compatibilitatea cu procesoarele pe 16 bit, dar apar instrucțiuni noi, precum și concepte noi, cum ar fi modul virtual de operare.

Unul din principalele dezavantaje ale sistemelor pe 32 bit este faptul că nu pot adresa mai mult de 4GB de memorie RAM (un sistem pe 16 bit putea adresa maxim 1MB de memorie convențională). Aplicațiile moderne, ce procesează volume mari de date au nevoie de spații de memorie mai largi. Au apărut astfel procesoarele pe 64 bit, care pot adresa până la 256 TB (se folosesc doar 48, din cei 64 bit pentru adrese).

2.1.3 Arhitectura setului de instrucțiuni

În continuare, se vor discuta componentele procesorului, vizibile pentru un programator în limbaj de asamblare. Astfel, un procesor dispune de un set de regiștri interni folosiți pentru păstrarea temporară a datelor, a adreselor sau a instrucțiunilor. Există regiștri generali folosiți în majoritatea operațiilor aritmetico-logice și regiștri speciali, care au o destinație specială.

Procesoarele x86 conțin 8 regiștri de uz general:

EAX – Accumulator

ESI – Source Index

EBX – Base

EDI – Destination Index

ECX – Counter

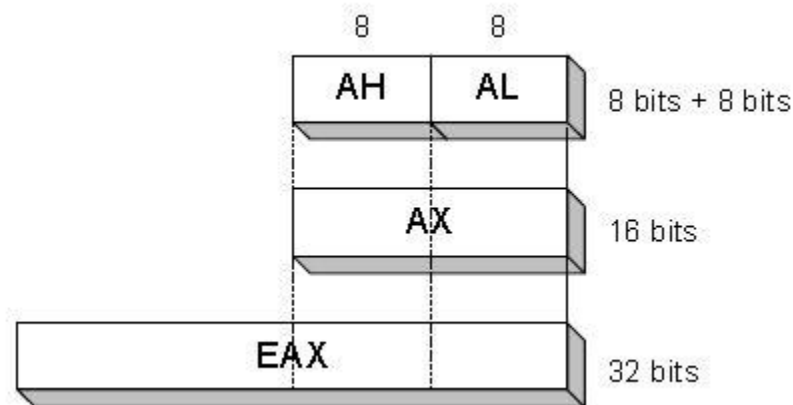
EBP – Base Pointer

EDX – Data

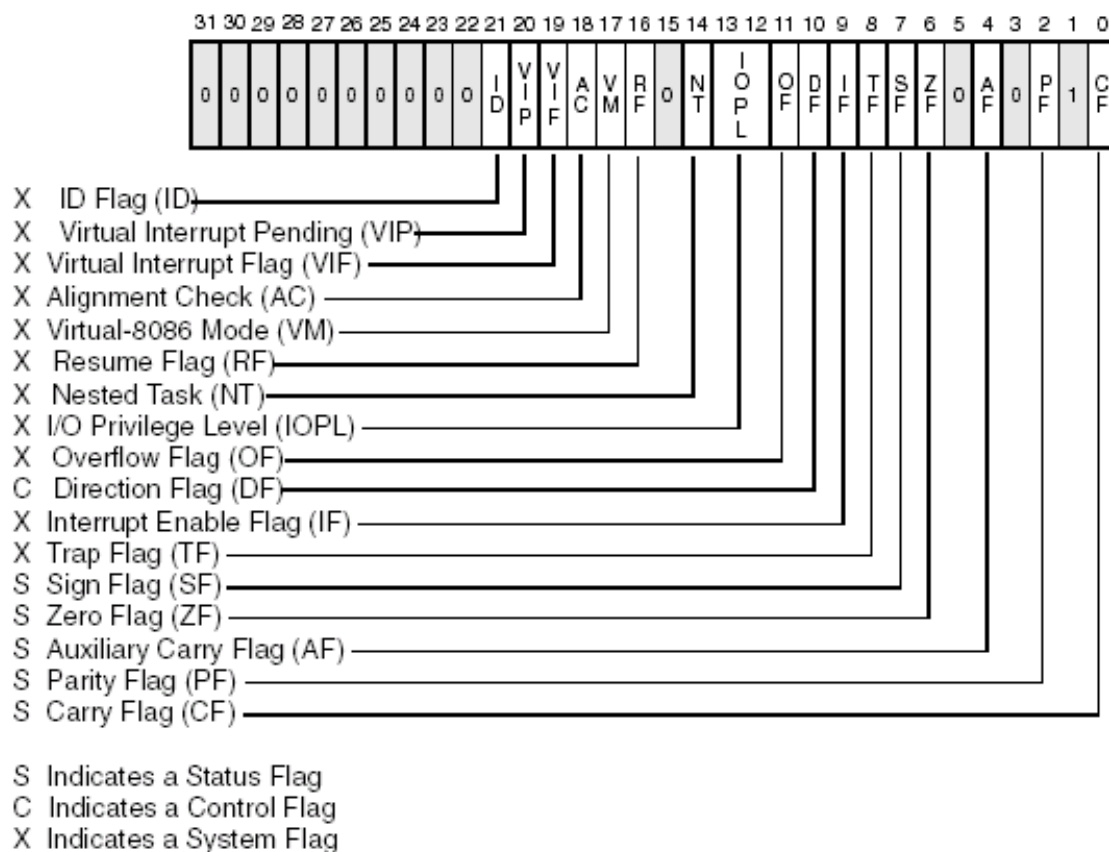
ESP – Stack Pointer

Fiecare dintre aceștia poate stoca o valoare pe 32 bit. Denumirea acestor regiștri începe cu litera 'E' (de la "Extended"), deoarece sunt extensii ale regiștrilor de uz general pe 16 bit: AX, BX, CX, DX, SI, DI, BP, SP. Dacă se doresc doar cei mai puțin semnificativi 16 bit dintr-un astfel de registru, se pot folosi aceste denumiri pe 16 bit. În plus, pentru primii 4 dintre acești regiștri se pot adresa separat cei mai puțin semnificativi 2 octeți, adică biții 0..7, respectiv 8..15, folosind

denumirile *L, respectiv *H, unde * poate fi una din literele A, B, C sau D. În figura de mai jos sunt exemplificate părțile adresabile ale registrului EAX.



Un alt registru important este EFLAGS, în care sunt reținuți diverși indicatori de stare sau control ai sistemului.



O parte din aceste flag-uri sunt:

- CF – Carry Flag – este setat pe 1, dacă la operația anterioară, s-a generat un transport

- PF – Parity Flag – indică paritatea rezultatului generat anterior
- AF – Auxiliary Carry Flag – indică faptul că la operația anterioară, pe cei mai puțin semnificativi 4 biți, s-a generat un transport
- ZF – Zero Flag – se setează dacă rezultatul operației anterioare este 0
- SF – Sign Flag – indică semnul rezultatului generat anterior
- OF – Overflow Flag – indică o depășire de capacitate, la ultima operație aritmetică

Registrul EIP (Instruction Pointer), pe 32 bit, indică locația în memorie a următoarei instrucțiuni ce va fi executată. Acest registru se mai numește uneori și PC (Program Counter). EIP nu poate fi modificat direct, precum regiștrii de uz general, dar se modifică indirect, prin instrucțiuni de control al fluxului de instrucțiuni.

Regiștrii segment (denumiți selectori, în asamblarea pe 32 bit) sunt regiștri pe 16 bit, și participă la formarea unei adrese, sau pointează către o tabelă de descriptori. Acești regiștri sunt: CS (Code Segment), DS (Data Segment), SS (Stack Segment), ES (Extended Segment), FS, respectiv GS.

Procesorul știe să execute diverse tipuri de instrucțiuni, având ca operanzi regiștri, adrese sau date din memoria principală, sau date imediate:

- instrucțiuni aritmetice și logice

Spre exemplu, `ADD EAX, EBX` adună valoarea din registrul EBX, la cea din registrul EAX, registrul EAX conținând rezultatul obținut. `SUB AH, 5` va scădea 5 din valoarea conținută în registrul AH.

- instrucțiuni de transfer a datelor

Sunt folosite pentru a copia date dintr-un registru în altul, sau din/în memorie. `MOV EDX, EAX` va copia conținutul registrului EDX în EAX. `MOV ECX, [123456H]` va copia în registrul ECX valoarea aflată în memoria principală la adresa 123456H.

- instrucțiuni de control al fluxului de instrucțiuni

Se folosesc atunci când se dorește ca următoarea instrucțiune executată să fie alta decât următoarea instrucțiune din program. De exemplu `JZ 100` (Jump if Zero) va sări peste următorii 100 de octeți din program, dacă rezultatul operației anterioare a fost 0, altfel va executa instrucțiunea următoare.

2.1.4 Organizarea memoriei

La organizarea memoriei, se întâlnesc 2 moduri de adresare: modul real și modul protejat:

În modul real spațiul maxim de adresare al memoriei este de 1MB. Această memorie este împărțită în segmente de lungime fixă de 64KB. Adresa de început a unui segment se păstrează în unul dintre regiștrii segment (CS, DS, SS, ...). Deoarece un registru segment are doar 16 bit, în el se păstrează doar partea mai semnificativă a adresei de segment, ultimii 4 bit considerându-se în mod implicit 0. Adresa unei locații de memorie se calculează ca o sumă între adresa de segment și o adresă de offset. Adresa de segment se obține prin multiplicarea conținutului registrului segment cu 16 (deplasarea la stânga cu 4 poziții binare). Adresa de offset se calculează pe baza modului de adresare și eventual a adresei conținute în codul de instrucțiune. Prin adunare se obține o adresă fizică pe 20 bit, suficientă pentru adresarea unui spațiu de 1 MB ($1M=2^{20}$). În exemplul de mai jos, pentru claritate, valorile de adrese sunt exprimate în hexazecimal.

1	2	3	4	0	Adresa de segment
	5	6	7	8	Adresa de offset
1	7	9	B	8	Adresa fizică

Acest mod de calcul a adresei fizice are câteva consecințe:

- spațiul maxim de adresare este 1MB
- un segment trebuie să înceapă la o adresa multiplu de 16
- un segment are maxim 64KB
- segmentele se pot suprapune parțial sau total
- aceeași locație fizică se poate exprima prin mai multe variante de perechi de adrese (segment:offset)
- există puține posibilități de protejare a zonelor de memorie
- orice program poate adresa orice locație de memorie, neputându-se impune restricții (lucru nedorit într-un sistem multitasking)

Modul protejat s-a introdus odată cu procesorul '386 și apoi s-a perfecționat la procesorul '486. Acest mod a fost necesar pentru a soluționa limitările modului real, în special în ceea ce privește spațiul limitat de adresare și posibilitățile reduse de protecție.

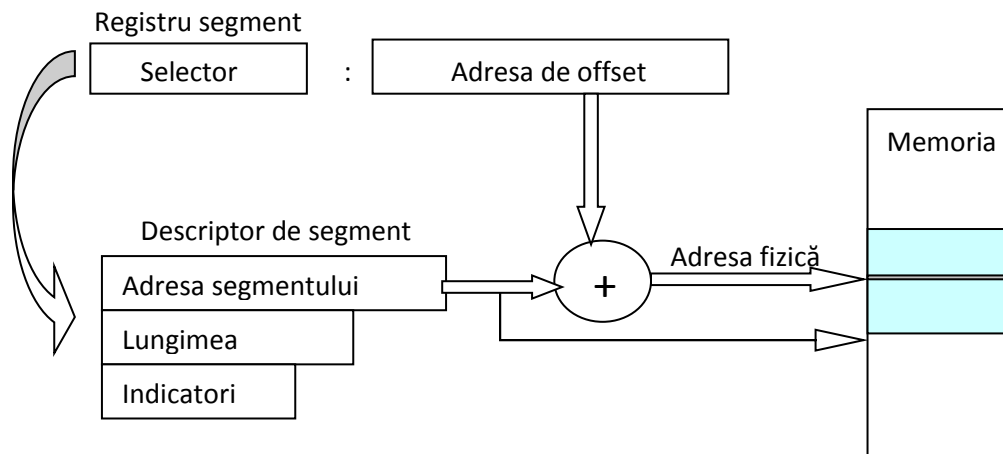
În modul protejat exprimarea adresei se face la fel prin adresă de segment și adresa de offset, însă cu anumite amendamente:

- un registru segment păstrează un selector de segment și nu adresa de început a segmentului;
- selectorul este un indicator care arată locul unde se află o structură de date care descrie un segment și care poartă numele de descriptor de segment

- un descriptor de segment conține: adresa segmentului (pe 32 de biți) lungimea segmentului (pe 20 de biți), indicatori pentru determinarea drepturilor de acces și indicatori care arată tipul și modul de utilizare a segmentului
- adresa de offset se exprimă pe 32 de biți

Aceste modificări generează următoarele consecințe:

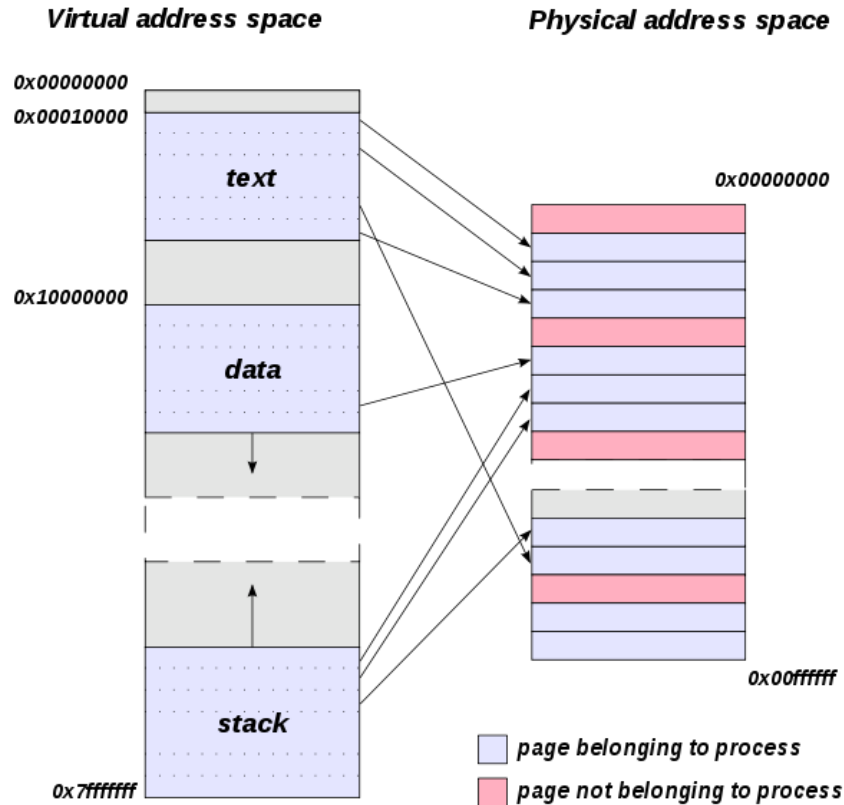
- spațiul maxim de adresare al memoriei se extinde la 4GB ($4G = 2^{32}$)
- un segment are o lungime variabilă, în interval larg de la 1 octet la 4GB
- se definesc trei nivele de protecție (0, cel mai prioritar)
- un segment este accesibil numai taskului alocat și eventual sistemului de operare
- anumite segmente pot fi blocate la scriere (ex: segmentele de cod)
- rezultă un mecanism complex de alocare și de protecție a zonelor de memorie



Calculul adresei fizice în modul protejat

În sistemele de operare Windows și Linux, pe 32 bit, s-a ales modelul memoriei "flat". Acest lucru înseamnă că în cadrul programului, toți descriptorii de segment, cu excepția segmentului FS, descriu un segment a cărui adresă de început este 0, iar lungimea este $2^{32}-1$. Segmentul FS este un segment special, care în sistemul de operare Windows conține structura TIB (Thread Information Block), și are altă adresă de început și altă lungime.

Fiecare program are acces la o memorie virtuală de 4GB (din care doar prima jumătate este adresabilă în mod utilizator), în care este izolat de celelalte programe. Acest lucru nu înseamnă că fiecare program folosește 4 GB de memorie fizică. Sistemul menține o tabelă cu corepondența dintre paginile de memorie virtuală (specifice programului) și cadrele de memorie fizică (specifice întregului sistem).



Astfel, fiecare program are propriul spațiu de adrese. Dacă un program scrie o dată la adresa 123456H în memorie, iar un alt program, ce rulează în paralel cu primul scrie altă valoare, la aceeași adresă, cele două valori nu se vor suprascrie una pe alta. Deși ambele programe scriu la adresa virtuală 123456H, în memoria fizică, cele două adrese virtuale vor avea corespondente diferite.

2.2 Elementele de bază ale limbajului de asamblare

2.2.1 Construcții de bază

O **instrucțiune** în limbaj de asamblare are următorul format:

```
[<etichetă>:] [<operație> [<operandi>]] [<comentarii>]
```

<etichetă> este un nume format din litere, cifre sau caractere speciale, care începe neapărat cu o literă sau cu un caracter special. Etichetele sunt folosite pentru a ne referi la o anumită poziție, din cadrul programului.

<operație> este mnemonica unei instrucțiuni în asamblare.

<operandi> pot fi zero, unu sau mai mulți, și reprezintă entitățile asupra cărora se efectuează operația. Pot fi regiștiri, locații de memorie, date imediate sau constante.

<comentarii> sunt orice text, ce începe cu ';', și continuă până la sfârșitul rândului. Acestea sunt ignorate de către asamblor, și utile pentru lizibilitatea codului.

Constantele numerice sunt specificate printr-un șir de cifre și litere, ce începe obligatoriu cu o cifră. Un număr zecimal se specifică pur și simplu printr-un șir de cifre. Un număr binar se specifică printr-un șir de cifre binare, terminate cu litera 'B'. Analog se folosește litera 'Q' pentru octal, respectiv 'H' pentru hexazecimal. Dacă un număr hexazecimal începe cu o literă (A..F), aceasta trebuie precedată de cifra 0.

Exemple: 010010100B, 26157Q, 4921, 4B52H, 0AB3H.

Caracterele sau șirurile de caractere se scriu între apostroafe (') sau ghilimele("). În mod intern, un șir de caractere este echivalent cu un șir de numere, pe 8 bit.

Simbolurile reprezintă poziții în memorie. Acestea pot fi etichete sau variabile.

Etichetele pot fi definite în zona de cod, și sunt folosite pentru a indica o anumită linie de cod. Pentru programator, e mai ușor să indice un salt la o anumită instrucțiune, decât să indice adresa, sau deplasamentul față de adresa curentă.

Variabilele sunt definite în zona de date. Acestea trebuie să aibă unul din tipurile simple: BYTE(1 octet), WORD(2 octeți), DWORD(4 octeți), QWORD(8 octeți), TWORD(10 octeți) sau un tip compus, STRUC sau RECORD.

Pentru a defini o variabilă, se scrie numele acesteia, urmat de un cuvânt cheie, care indică tipul variabilei: DB (de la Define Byte) pentru BYTE, DW pentru WORD, DD pentru DWORD, etc. Pentru a scrie un vector în locul unei variabile simple, valorile se pot scrie, separate de virgulă:

```
vect DB 1, 2, 3, 4, 5
```

Pentru a defini <n> elemente, cu valoarea <x>, se poate scrie <n> DUP(<x>), de exemplu:

```
vect DB 5 DUP(0)
```

Pentru a defini o **constantă**, se folosește cuvântul cheie EQU, într-o expresie de genul:

```
<nume_const> EQU <expresie>
```

De exemplu, se poate scrie

```
ZECE EQU 10
```

Valoarea constantei nu se va stoca în memoria programului, ci asamblorul va înlocui în codul sursă numele acesteia, cu valoarea (similar cu #define din C, nu cu const).

2.2.2 Structura generală a unui program MASM

Un program MASM, are următoarea structură:

1	.386
2	.model flat, stdcall
3	;;;
4	
5	;includem biblioteci, si declarăm ce funcții vrem să importăm
6	includelib msvcrt.lib
7	extern exit: proc
8	;;;
9	
10	;declaram simbolul start ca public - de acolo începe execuția
11	public start
12	;;;
13	
14	;secțiunile programului, date, respectiv cod
15	.data
16	;aici declaram date
17	
18	.code
19	start:
20	;aici se scrie codul
21	
22	;terminarea programului
23	push 0
24	call exit
25	end start

Pe prima linie a programului întâlnim directiva `.386`, care spune asamblorului să genereze un fișier binar, compatibil cu procesoarele din generația '386. Linia a doua conține directiva `.model`, prin care se specifică tipul memoriei, respectiv tipul limbajului. Modelul `flat` este cel folosit în Windows și implică folosirea întregului spațiu de adrese, pentru toate selectoarele, cu excepția selectorului FS. `stdcall` reprezintă convenția de apel a funcțiilor și a altor simboluri exportate de program.

Linia 6 conține directiva `includelib`, prin care se include în programul nostru bibliotecă `msvcrt`. Această bibliotecă conține funcții și proceduri standard, cum ar fi `printf`, `exit`. Pentru a folosi o astfel de funcție, aceasta trebuie declarată ca simbol extern, după cum se observă în linia 7.

Atunci când un program rulează, sistemul de operare trebuie să știe de unde să înceapă execuția. Din acest motiv, trebuie să marcăm acest lucru în mod explicit, în programul nostru, printr-o etichetă (linia 19), apoi să exportăm această etichetă ca simbol public (linia 11).

Directiva `.data` de la linia 15 marchează începutul secțiunii de date. În această secțiune se pot declara variabilele și constantele folosite de program. Analog, directiva `.code` de la linia 18 marchează începutul secțiunii de cod. Acolo se vor scrie toate instrucțiunile programului.

2.3 Instrumentele de lucru pentru compilare și depanare

2.3.1 Asamblorul și linker-ul

Pentru ca un program scris în limbaj de asamblare să se transforme în cod binar, care va fi executat direct de către procesor, este nevoie de un program special numit asamblor. În cadrul acestui laborator, asamblorul folosit este MASM (Microsoft Macro Assembler).

În directorul `masm_minimal`, se găsește fișierul `ml.exe`, cu ajutorul căruia se assemblează fișierele sursă. Pentru asamblare, se tastează următoarea comandă, în Command Prompt:

```
>ml.exe sursa.asm
```

unde `sursa.asm` este numele fișierului ce conține codul sursă. Rezultatul acestui apel nu generează un fișier executabil, ci un fișier obiect, ce conține codul binar, dar nu poate rula. Pentru a se obține un fișier executabil, trebuie editate legăturile programului. Pentru această operație, există fișierul `link.exe` din folderul `masm_minimal`. Asamblarea și link-editarea se pot face printr-o singură comandă:

```
>ml.exe sursa.asm /link /subsystem:console /entry:start msvcrt.lib
```

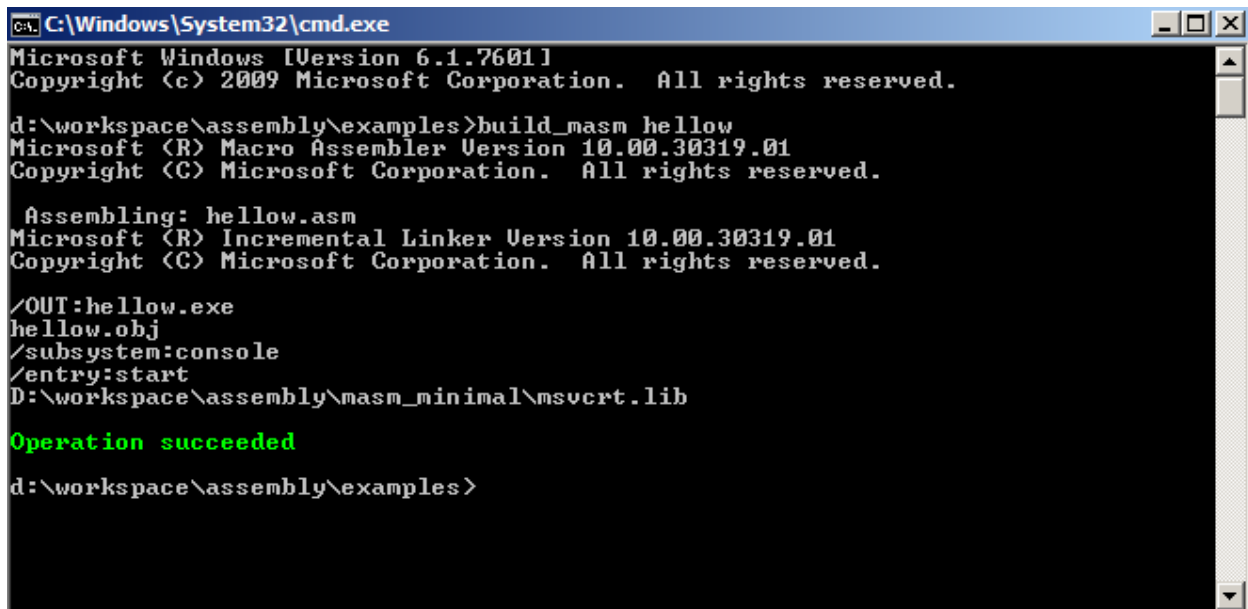
Această comandă assemblează fișierul `sursa.asm`, generând programul de consolă `sursa.exe`, ce are codul de la eticheta `start`, ca punct de pornire, și include biblioteca `msvcrt.lib`.

Pentru a nu fi necesară scrierea acestei comenzi de fiecare dată, în `masm_minimal` se găsește script-ul `build_masm.bat`, care primește ca parametru numele sursei, fără extensie, și încearcă să genereze programul executabil.

Pentru a asambla și link-edita fișierul `hellow.asm`, și genera `hellow.exe`, se folosește comanda:

```
>build_masm hellow
```

Rezultatul comenzii, se poate observa în figura de mai jos:



```
C:\Windows\System32\cmd.exe
Microsoft Windows [Version 6.1.7601]
Copyright (c) 2009 Microsoft Corporation. All rights reserved.

d:\workspace\assembly\examples>build_masm hellow
Microsoft (R) Macro Assembler Version 10.00.30319.01
Copyright (C) Microsoft Corporation. All rights reserved.

  Assembling: hellow.asm
Microsoft (R) Incremental Linker Version 10.00.30319.01
Copyright (C) Microsoft Corporation. All rights reserved.

/OUT:hellow.exe
hellow.obj
/subsystem:console
/entry:start
D:\workspace\assembly\masm_minimal\msvcrt.lib

Operation succeeded

d:\workspace\assembly\examples>
```

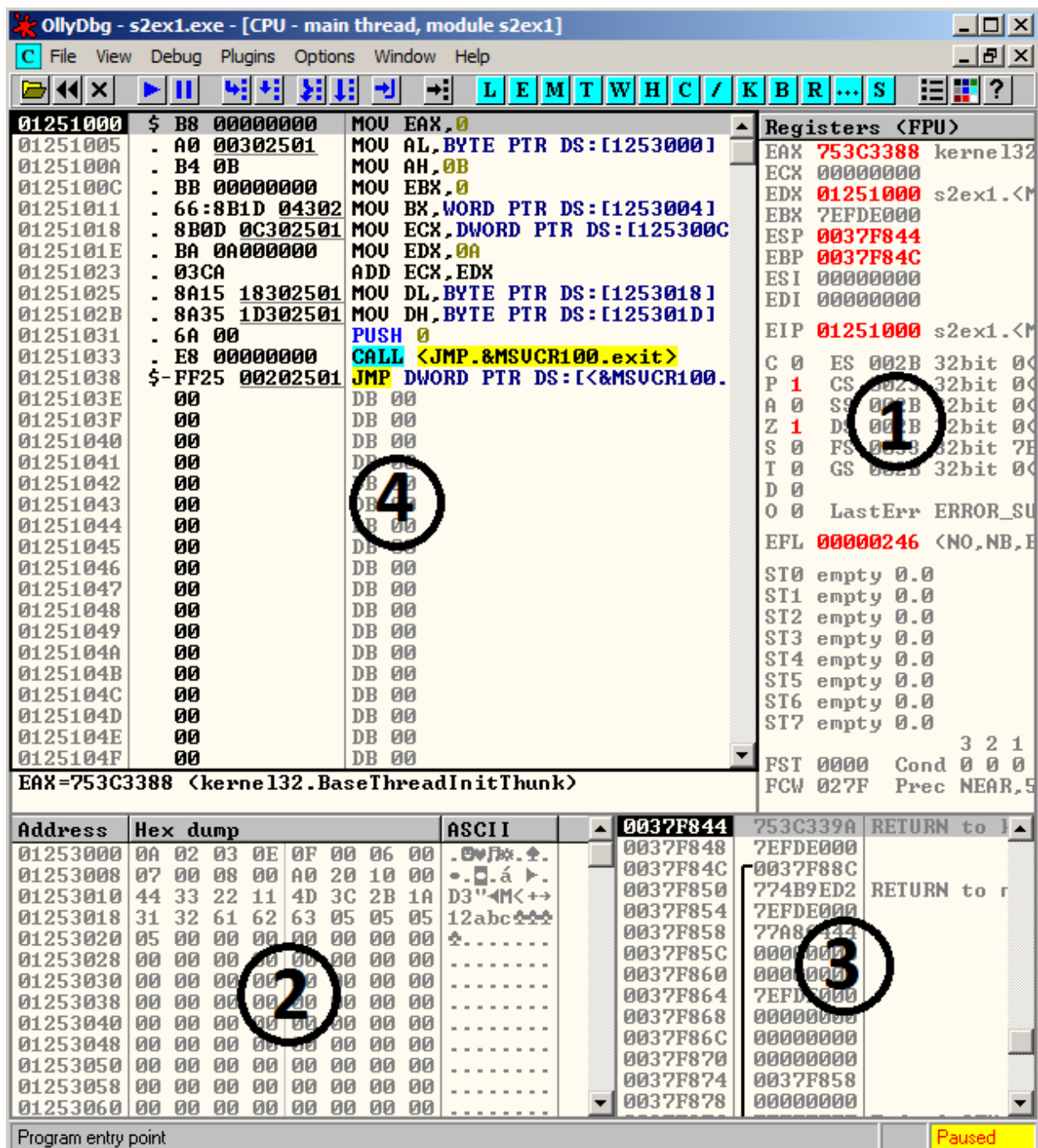
În caz de eroare, în linia de comandă se va afișa eroarea, și linia din codul sursă la care s-a produs eroarea respectivă. Se recomandă folosirea unui editor special pentru a scrie codul sursă (de exemplu, Notepad++), care numerotează liniile de cod.

2.3.2 Depanarea programelor

Pentru a putea înțelege mai bine execuția unui program în limbaj de asamblare, sau pentru depistarea erorilor, se folosesc programe de tip debugger. Acestea permit încărcarea unui fișier executabil, execuția acestuia instrucțiune cu instrucțiune, vizualizarea conținutului memoriei și a regiștrilor, la fiecare pas, și chiar modificarea unor instrucțiuni sau date, în timp ce programul rulează.

Debugger-ul folosit la acest laborator va fi Olly Debugger. Lansarea acestuia în execuție se face făcând dublu-click pe executabilul acestuia, numit "OLLYDBG.EXE".

Încărcarea unui program pentru depanare, se va face folosind tasta F3. Eventualele argumente cu care trebuie rulat programul se pot specifica în fereastra de dialog care apare. Dacă după încărcarea programului nu apare codul acestuia, se poate ajunge la el, folosind combinația de taste Ctrl+F9.



Fereastra debugger-ului arată ca în figura de mai sus, și este împărțită în 4 zone:

Zona 1 – conține regiștrii procesorului, împreună cu valorile acestora, la momentul curent al execuției. Regiștrii ai căror valori s-au modificat la instrucțiunea anterioară sunt marcați cu roșu, iar restul cu gri. Flag-urile sunt afișate și separat. Tot în această zonă sunt afișați și regiștrii coprocesorului matematic.

Zona 2 – numită și dump, poate afișa diverse porțiuni din memoria programului. La începutul execuției, aici se afișează secțiunea de date. Dump-ul este afișat prin 3 coloane. În prima avem adresa de început a liniei, în a doua, conținutul memoriei la adresa respectivă în format hexazecimal, iar în a 3-a, același conținut, în format text. Dacă dorim să vedem conținutul memoriei de la o anumită adresă, care apare în una din cele 4 zone ale debugger-ului, se face click dreapta pe acea adresă, și din meniul apărut se alege opțiunea “Follow in Dump”.

Zona 3 – conține stiva programului (zonă de memorie cu o întrebuințare specială, ce va fi dezbătută într-o lucrare viitoare). Pe prima coloană sunt afișate adresele, iar pe a doua valorile, în hexazecimal (câte un DWORD). Adresa vârfului stivei este evidențiată. Se poate observa că adresa vârfului stivei este valoarea din registrul ESP. Ca și pentru zona de dump, orice adresă ce aparține de stivă poate fi urmărită în aceasta, făcând click dreapta și alegând “Follow in Stack”.

Zona 4 – este zona de afișare a codului. În prima coloană se găsesc adresele instrucțiunilor, în hexazecimal. Adresa instrucțiunii următoare este evidențiată. În a doua coloană se găsește codul binar, aferent instrucțiunii de la adresa respectivă. Codurile binare ale instrucțiunilor au lungimi variabile. Pe a 3-a coloană se regăsește codul programului, dezasamblat. Ar trebui să se observe aceleași instrucțiuni ca și în programul sursă, eventual scrise într-o altă ordine.

Comenzi pentru depanare:

- Step Into – **F7** – trece la instrucțiunea următoare a programului, intrând în funcții, acolo unde se întâlnesc.
- Step Over – **F8** – trece la instrucțiunea următoare, sărind peste funcții (se execută întreaga funcție, ca și cum ar fi o singură instrucțiune).
- Breakpoint – **F2** – atunci când cursorul se află pe o anumită linie (aceea linie este evidențiată prin culoarea gri deschis), se plasează o întrerupere la acea linie. Atunci când programul, în timpul rulării, ajunge la o instrucțiune pe care s-a pus un breakpoint, se va întrerupe execuția.
- Run – **F9** – pornește execuția normală a programului, de la poziția curentă, până la primul breakpoint întâlnit, sau până la final.
- Execute till Return – **Ctrl+F9** – la fel cu Run, dar execuția se oprește și la întâlnirea unei instrucțiuni RETN.
- Restart – **Ctrl+F2** – repornește programul depanat.

Instrucțiunile sau datele unui program pot fi modificate în timpul depanării. Se face click pe instrucțiunea sau datele dorite, pentru a poziționa cursorul acolo, apoi se apasă tasta **Space**. În fereastra de dialog apărută, se pot face modificările.

Pentru a vizualiza întreg conținutul memoriei, se apasă combinația de taste **Alt+M**. În tabelul apărut este descrisă fiecare secțiune. Coloanele Address și Size indică adresa, respectiv

dimensiunea secțiunii. În coloana owner, apare numele modulului ce conține acea secțiune. Modulul principal are același nume ca și numele programului executabil. Coloana Section indică numele efectiv al secțiunii, în timp ce coloana Contains indică ce se găsește în aceasta.

Făcând dublu-click pe o linie a tabelului, se poate vizualiza conținutul secțiunii respective. Olly Debugger va încerca să ”ghicească” tipul de conținut, afișând implicit datele în format hexazecimal și ASCII, respectiv dezamblând codul. În cazul în care utilizatorul dorește vizualizarea informației în alt format, poate face click dreapta în zona de afișare, și alegerea formatului dorit.

Memory map

Address	Size	Owner	Section	Contains	Type	Acc
0030D000	00003000			stack of main thread	Priv 00021104	RW
00370000	00003000				Priv 00021104	RW
003C9000	00007000				Priv 00021104	RW
00440000	00003000				Priv 00021104	RW
00450000	00007000				Priv 00021104	RW

Dump - MSVCR100.data 6B271000..6B276FFF

Address	Size	Owner	Section	Contains	Type	Acc
10020000	00002000	leaktest	.reloc	relocations	Imag 01001002	R
4C000000	00001000				Priv 00021040	RWE
6B1C0000	00001000	MSUCR100		PE header	Imag 01001002	R
6B1C1000	0000B0000	MSUCR100	.text	code, imports, exports	Imag 01001002	R
6B271000	00006000	MSUCR100	.data		Imag 01001002	R
6B277000	00001000	MSUCR100	.rsrc	resources	Imag 01001002	R
6B278000	00005000	MSUCR100	.reloc	relocations	Imag 01001002	R
70C10000	00001000	plugin_f		PE header	Imag 01001002	R
70C11000	00016000	plugin_f	.text	code	Imag 01001002	R
70C27000	00010000	plugin_f	.rdata	imports, exports	Imag 01001002	R
70C37000	00079000	plugin_f	.data	data	Imag 01001002	R
70C38000	00001000	plugin_f	.rsrc	resources	Imag 01001002	R

Pentru revenirea în mod CPU (fereastra principală, cu cele 4 zone), se apasă combinația de taste **Alt+C**.

2.4 Mersul lucrării

- g. Se va urmări modul în care sunt păstrate datele în memorie. Fiecare patrat de mai jos reprezintă un octet din zona de date a programului s2ex1.exe. Să se completeze fiecare octet cu conținutul corespunzător și să se delimiteze variabilele declarate.

[illegible]